Algoritmos e Estruturas de Dados III - Árvores B

Paulo Lisboa de Almeida

1º Semestre - 2026

1 Descrição

O programa a seguir servirá como base para a prova prática da disciplina. Caso você não traga o programa, ou se o programa não for de sua autoria (por exemplo, por plágio ou uso de IA Generativa), sua prova será anulada. Além das questões que serão reveladas apenas durante a prova, a qualidade do seu programa terá grande peso na avaliação.

O Objetivo é implementar uma Árvore B utilizando a linguagem de programação C. A implementação deve seguir exatamente a descrita em Cormen et al. (algoritmos discutidos em sala). Você deve assumir que a árvore está na memória principal (não devem ser feitas leituras e escritas de dados no disco). Os demais detalhes são dados a seguir.

2 Template

Os algoritmos devem ser implementados utilizando como base o arquivo `arvoreB.h`, que contém os protótipos das funções que obrigatoriamente devem ser implementadas.

Não é permitida a mudança nos protótipos de funções disponibilizados no arquivo `arvoreB.h`. É permitida a criação de funções auxiliares se necessário. Também é permitida a criação de arquivos .h ou .c extras, mas tenha em mente que durante os testes, somente o arquivo `arvoreB.h` será incluído (no *main*, por exemplo).

Inclua um *Makefile*. Seu programa deve compilar sem erros ou avisos através do comando *make*. O comando make será executado a partir da raiz do seu projeto.

Durante a programação, é obrigatória a utilização do padrão C23 (`-std=c23`, disponível a partir do gcc-14). Caso você esteja usando uma máquina com versões antigas do gcc, faça o código de forma compatível com o seu compilador, e deixe comentada uma versão que é compatível com o C23.

O nome do binário gerado deve ser *prova1_GRR* (e.g., *prova1_1234*). Ajuste isso no *makefile*. Deve ser possível compilar e executar o binário em uma máquina Linux x86-64 com Ubuntu 24.04.

3 Detalhes de implementação

As seguintes regras devem ser obedecidas:

- A chave dos nodos deve ser um inteiro de 32 bits.

- Considere que o T da árvore pode ser, no máximo, $2^{30} - 1$. Dica: guarde em um int de 32 bits para facilitar, ao invés de usar size_t, que pode complicar a lógica.
- O programa deve compilar sem erros ou avisos através do comando *make*.
- O programa deve seguir boas práticas de programação em C, sendo portável (exemplo: não assuma tamanhos de variáveis que são específicos de um x86-64).
- Garanta que os custos dos algoritmos implementados é compatível com a teoria sobre árvores B.
- Ao faltar memória (em um `malloc`, por exemplo), o programa deve ser terminado imediatamente através da chamada `exit(1)`. Exiba a mensagem de erro "*Falha ao alocar memoria.*" no buffer `stderr`.
- Inclua um arquivo `main.c` com alguns testes do seu programa. Esse arquivo servirá como um backup caso, por exemplo, por erro de programação seu arquivo `arvoreB.h` não possa ser incluído no `main.c` de testes do professor. Nesse caso sua nota será, no melhor caso, parcial.

4 Qualidade do programa

Espere descontos severos por programas que não seguem boas práticas e afins. Exemplos de problemas incluem funções que exibem erros na tela ao invés de retornar códigos de erro, uso de tipos de dados incorretos, códigos obscuros e afins.

Vazamentos de memória e codificações com brechas de segurança (exemplo: *buffers oferflows* devido ao uso incorreto de `scanf` e afins) sofrerão os maiores descontos.

5 Descrição das Funções

```
struct arvoreB* crirArvoreB(int32_t t_arvore);
```

Aloca uma struct do tipo `arvoreB` com um valor de T específico e a retorna.

```
void inserirArvoreB(struct arvoreB* arvore, int32_t chave);
```

Insere a chave na Árvore B.

```
void imprimirArvoreB(struct arvoreB* arvore);
```

Imprime a Árvore B na tela em largura, seguindo estritamente o padrão do exemplo (não seguir o padrão invalidará sua função).

No exemplo, ao imprimir a lista de nodos do nível, faça no formato `F/I (n:NUMEROCHAVES) [chave0<espaço>chave1<espaço>...]<espaço><espaço>PROXIMONODO` onde F/I indica se o nodo é uma Folha ou um nodo Interno.

```
----//----
Nivel 0
----//----
I (n:1) [4]
----//----
Nivel 1
----//----
I (n:1) [2]     I (n:3) [6 8 10]
----//----
Nivel 2
----//----
F (n:1) [1]  F (n:1) [3]  F (n:1) [5]  F (n:1) [7]  F (n:1) [9]  F (n:2) [11 12]
```

```
void imprimirEmOrdem(struct arvoreB* arvore);
```

Imprime as chaves da árvore em ordem, seguindo o formato exemplificado a seguir:

```
Em ordem: 1 2 3 4 5 6 7 8 9 10 11 12
```

```
union ou struct nodo* buscarArvoreB(struct arvoreB* arvore, int32_t chave,
int32_t* idxEncontrado);
```

Retorna o endereço do nodo que contém a chave sendo buscada, e insere o índice onde essa chave se encontra dentro de `idxEncontrado`. Caso não encontre a chave, retorna `NULL` e insere -1 em `idxEncontrado`.

```
void deletarArvore(struct arvoreB* arvore);
```

Libera toda a memória alocada. Ou seja, todos os nodos são liberados, além da struct `arvoreB` passada como parâmetro.

6 Arquivos a serem entregues

Os arquivos devem estar em um diretório de nome *prova1_GRR*. Se, por exemplo, o grr do aluno é 1234, o diretório contendo os arquivos deve se chamar *prova1_1234*. Compacte esse diretório, sendo que a versão compactada vai se chamar *prova1_1234*.tar.gz. Ao descompactar o arquivo *prova1_1234*.tar.gz, deverá ser criado um diretório de nome *prova1_1234* que conterá todos os demais arquivos. O diretório deve conter o seguinte:

- arquivos de código fonte .c e .h do programa;
- makefile;

Não inclua quaisquer outros arquivos irrelevantes para o projeto. Por exemplo, não inclua binários compilados, ou arquivos objeto (.o). A inclusão de arquivos irrelevantes pode acarretar em descontos de nota.

7 Entrega

Submeta os arquivos na UFPRVirtual até um dia antes prova 1, quando os arquivos serão utilizados por você.

8 Distribuição da Nota

Alguns descontos padrão, considerando uma nota entre 0 e 100 pontos:

- Plágio ou uso de IA Generativa: perda total da pontuação para todos os envolvidos. Isso é válido mesmo para casos onde o plágio se refere a apenas um trecho do código.
- Entrega, mas não comparecimento no dia da prova, ou incapacidade de implementar os itens solicitados durante a prova, acarreta na perda total dos pontos.
- Algoritmos implementados em linguagem diferente de C serão desconsiderados.
- Não submissão via UFPRVirtual acarreta na perda total dos pontos.
- Falta de algum arquivo requisitado: desconto de 10 a 100 pontos.
- Inclusão de arquivos desnecessários (lixo): desconto de 5 a 20 pontos.
- Erros e avisos de compilação: desconto de 5 a 100 pontos.
- Nomes de arquivo incorretos: 10 pontos por arquivo.
- Arquivo com formato incorreto: 10 a 100 pontos.
- Problemas de programação, como uso de funções descontinuadas, programação insegura, e outros problemas: 10 a 100 pontos.
- Corrupção, vazamentos de memória, *user after free*, *double free*, entre outros problemas de gerenciamento de memória: 10 a 100 pontos.
- Formato de saída incorreto: 10 a 100 pontos.
- Algoritmos implementados incompatíveis com uma árvore B: 5 a 100 pontos.

9 Demais Regras

- Dúvidas ou casos não especificados neste documento podem ser discutidos com o professor até a **data de entrega**. **Não** serão aceitas reclamações após a data da entrega.
- Os alunos devem procurar o professor para tirar dúvidas até a data da prova.
- O descumprimento das regras dispostas nesse documento invalidará a prova.